

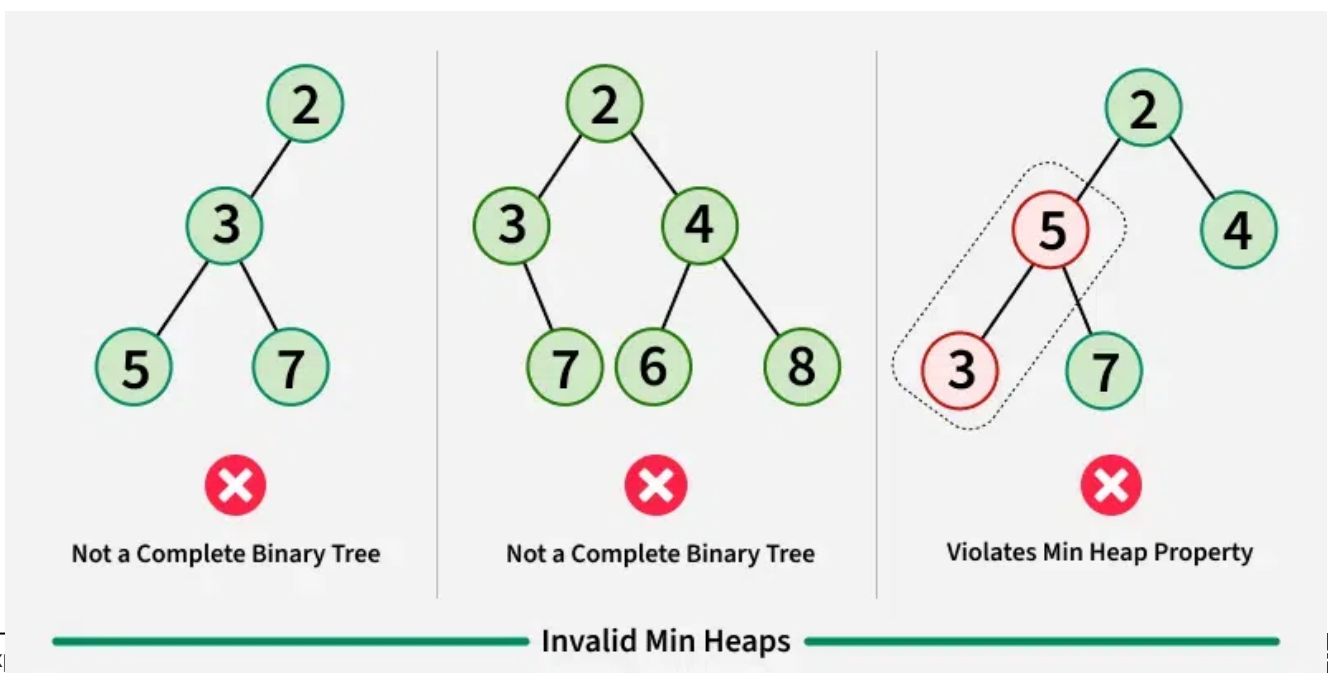
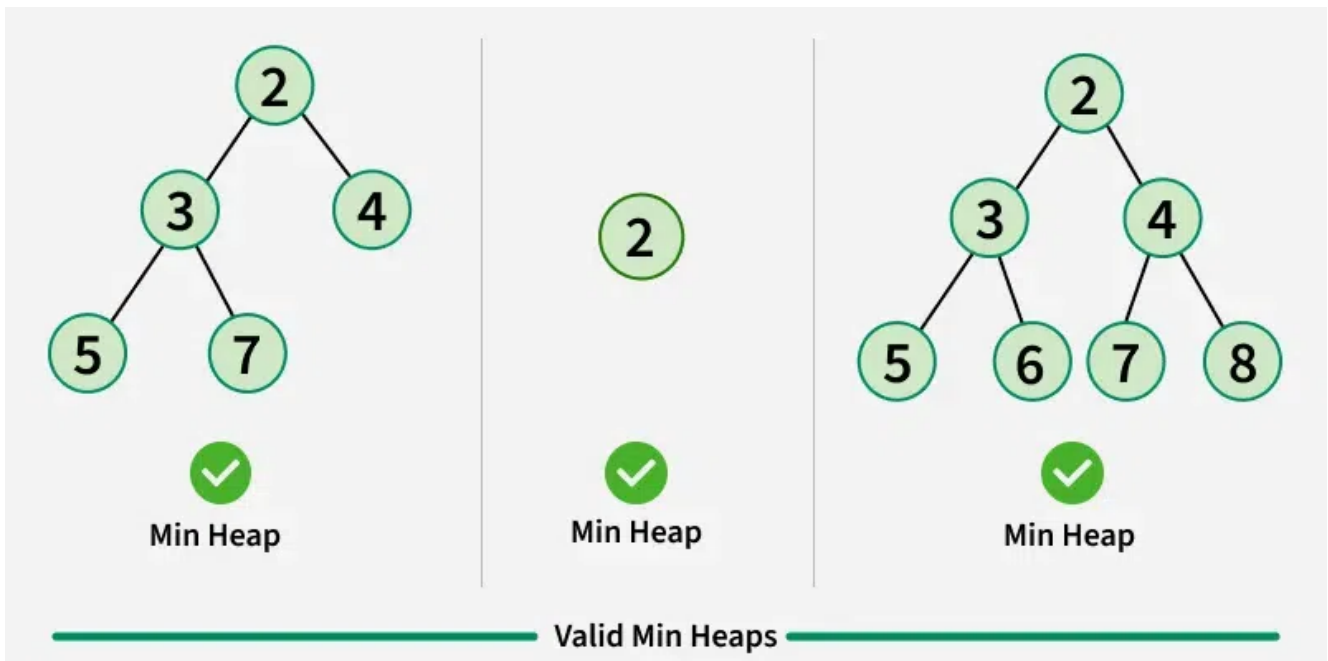
Binary Heap

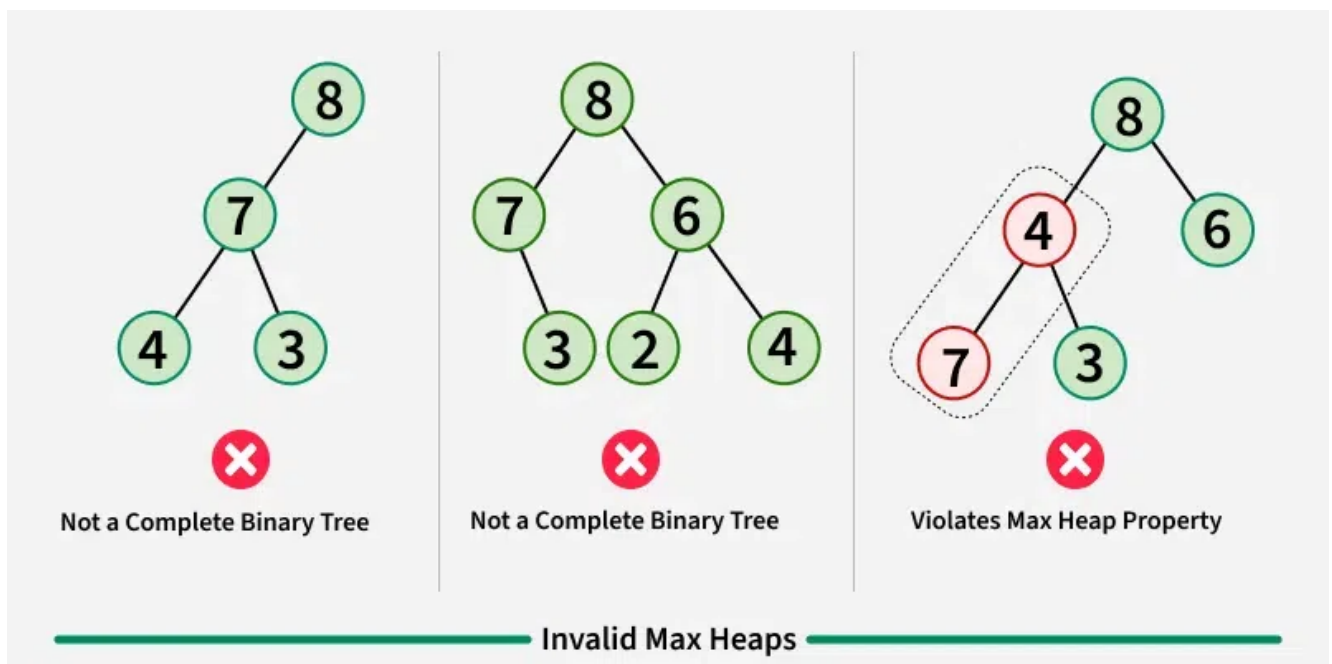
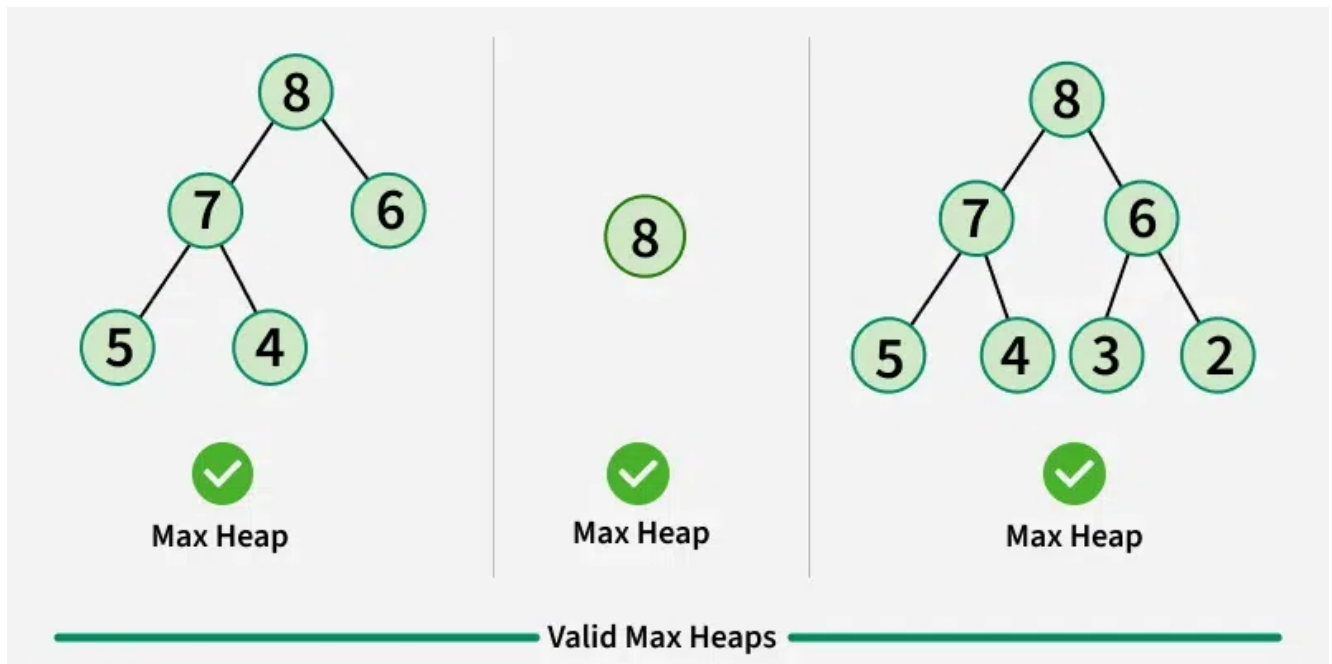
Last Updated : 24 Mar, 2025



A Binary Heap is a complete binary tree that stores data efficiently, allowing quick access to the maximum or minimum element, depending on the type of heap. It can either be a Min Heap or a Max Heap. In a Min Heap, the key at the root must be the smallest among all the keys in the heap, and this property must hold true recursively for all nodes. Similarly, a Max Heap follows the same principle, but with the largest key at the root.

Valid and Invalid examples of heaps





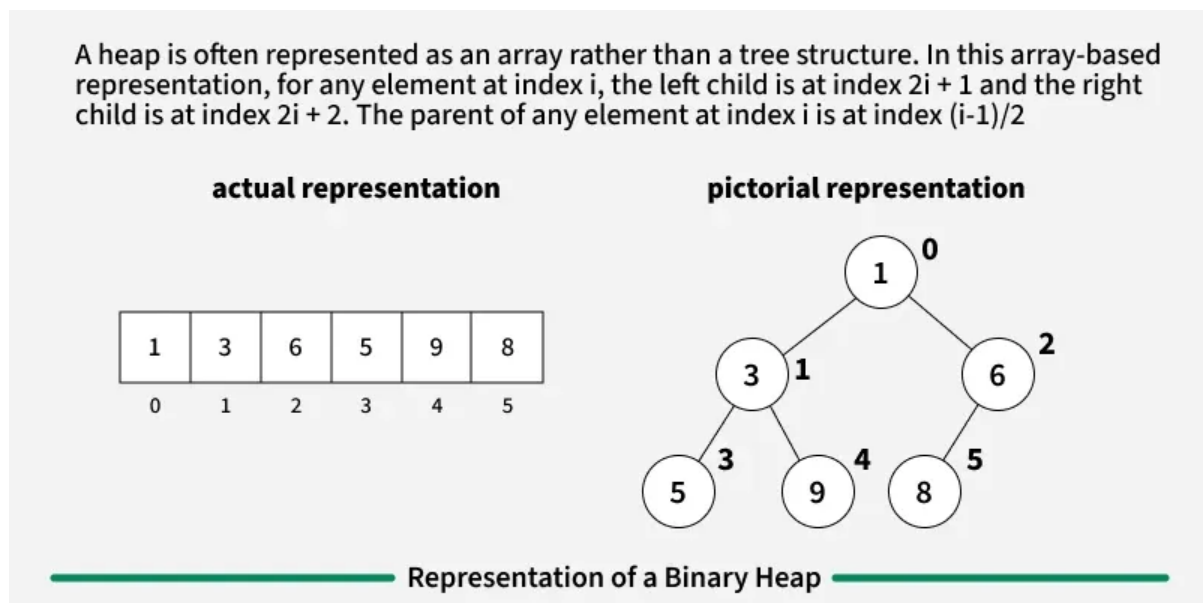
How is Binary Heap represented?

A Binary Heap is a **Complete Binary Tree**. A binary heap is typically represented as an array.

- The root element will be at `arr[0]`.
- The below table shows indices of other nodes for the i^{th} node, i.e., `arr[i]`:

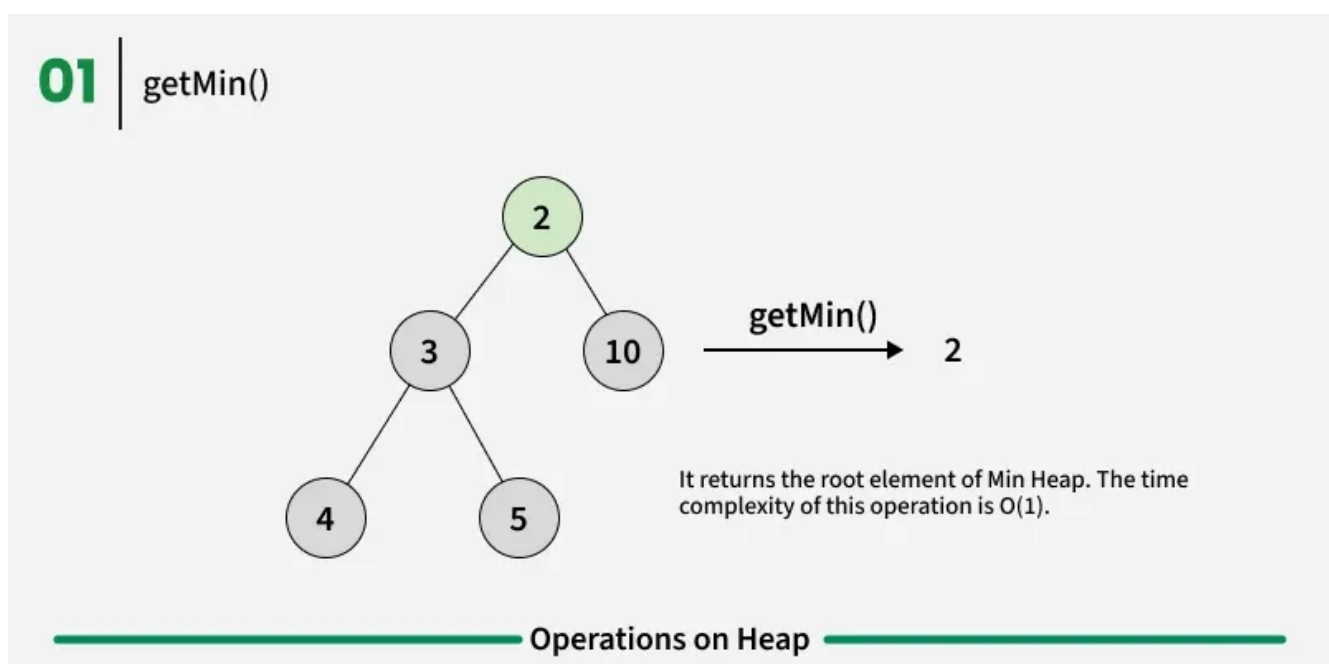
$\text{arr}[(i-1)/2]$	Returns the parent node
$\text{arr}[(2*i)+1]$	Returns the left child node
$\text{arr}[(2*i)+2]$	Returns the right child node

The traversal method use to achieve Array representation is [Level Order Traversal](#). Please refer to [Array Representation Of Binary Heap](#) for details.

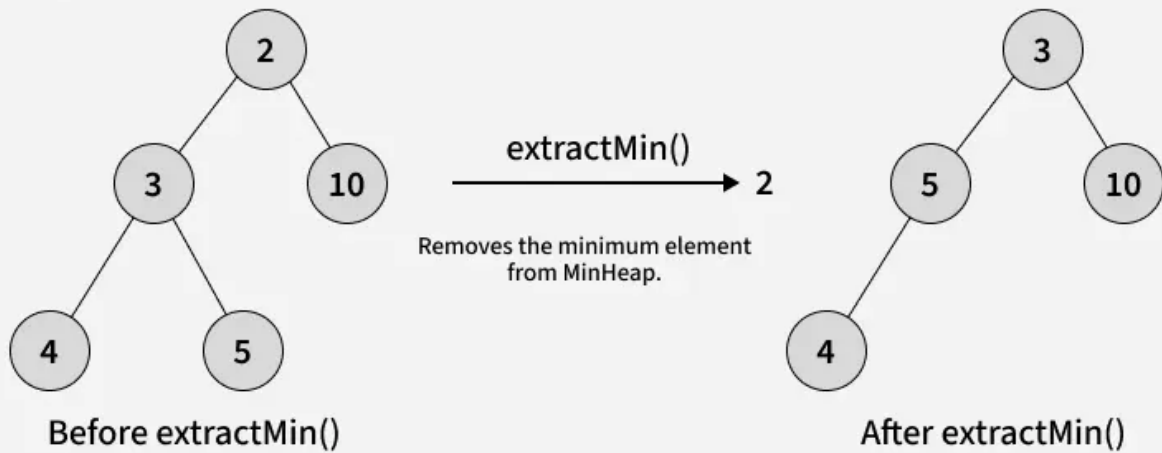


Representation of Binary Heap

Operations on Heap

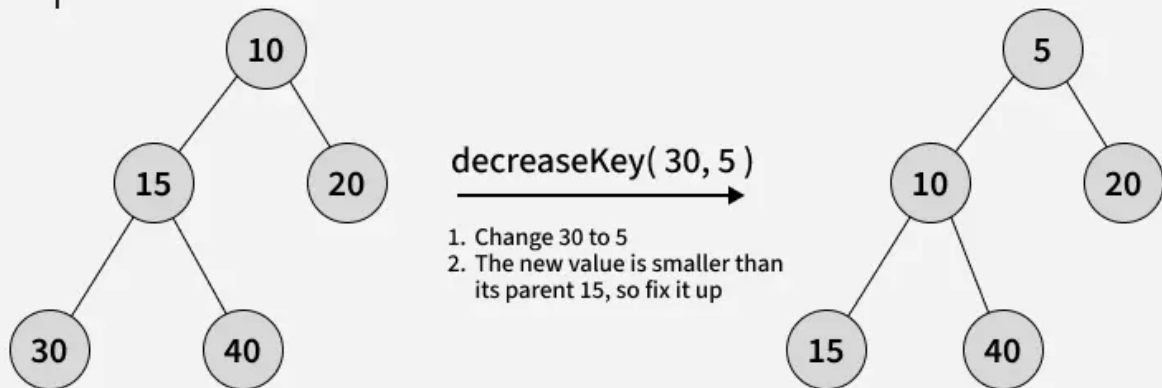


02 | extractMin()



Operations on Heap

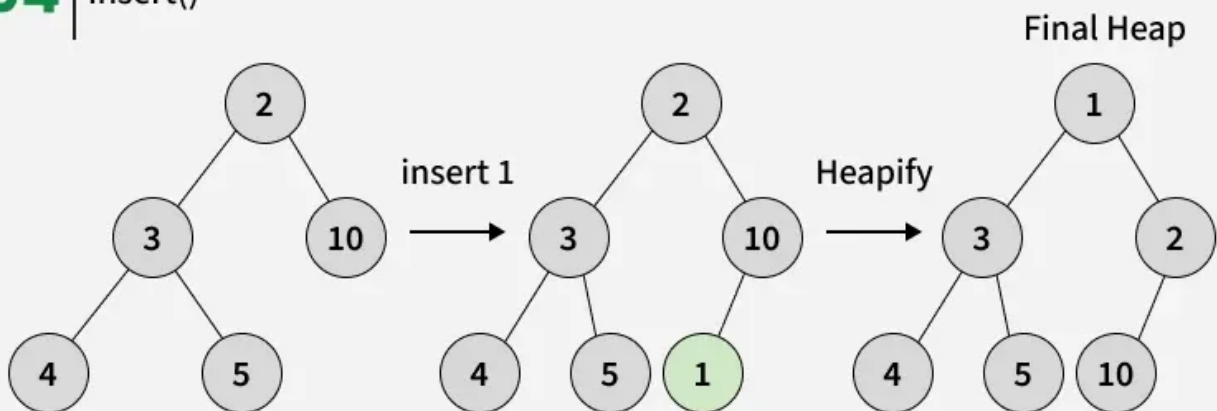
03 | decreaseKey()



Decrease the value of the key. If the decreased key value of a node is greater than the parent of the node, then we don't need to do anything. Otherwise, we need to traverse up to fix the violated heap property.

Operations on Heap

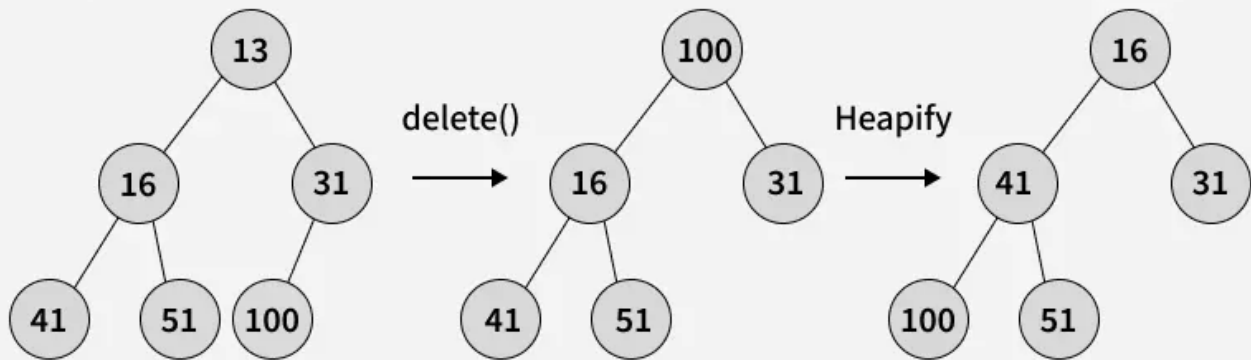
04 | insert()



Add a new key at the end of the tree. If the new key is greater than its parent, then we don't need to do anything. Otherwise, we need to traverse up to fix the violated heap property.

Operations on Heap

05 | delete()



Replace the root element with the last element in the heap and remove the last element. Since the new root may violate the heap property, heapify the root to restore the heap structure.

Operations on Heap

Refer [Introduction to Min-Heap – Data Structure and Algorithm Tutorials](#) for more

C++

Java

Python

C#

JavaScript

```
// A C++ program to demonstrate common Binary Heap Operatio
```



```
#include<iostream>
```

```
#include<climits>
```

```
using namespace std;
```

```
// Prototype of a utility function to swap two integers
```

```
void swap(int *x, int *y);
```

```
// A class for Min Heap
```

```
class MinHeap
```

```
{
```

```
    int *harr; // pointer to array of elements in heap
```

```
    int capacity; // maximum possible size of min heap
```

```
    int heap_size; // Current number of elements in min heap
```

```
public:
```

```
    // Constructor
```

```
    MinHeap(int capacity);
```

```
    // to heapify a subtree with the root at given index
```

```
    void MinHeapify(int i);
```

```
    int parent(int i) { return (i-1)/2; }
```

```
    // to get index of left child of node at index i
```

```
    int left(int i) { return (2*i + 1); }
```

```
    // to get index of right child of node at index i
```

```
    int right(int i) { return (2*i + 2); }
```

```
    // to extract the root which is the minimum element
```

```
    int extractMin();
```

```

// Decreases key value of key at index i to new_val
void decreaseKey(int i, int new_val);

// Returns the minimum key (key at root) from min heap
int getMin() { return harr[0]; }

// Deletes a key stored at index i
void deleteKey(int i);

// Inserts a new key 'k'
void insertKey(int k);
};

// Constructor: Builds a heap from a given array a[] of given size
MinHeap::MinHeap(int cap)
{
    heap_size = 0;
    capacity = cap;
    harr = new int[cap];
}

// Inserts a new key 'k'
void MinHeap::insertKey(int k)
{
    if (heap_size == capacity)
    {
        cout << "\n0verflow: Could not insertKey\n";
        return;
    }

    // First insert the new key at the end
    heap_size++;
    int i = heap_size - 1;
    harr[i] = k;

    // Fix the min heap property if it is violated
    while (i != 0 && harr[parent(i)] > harr[i])
    {
        swap(&harr[i], &harr[parent(i)]);
        i = parent(i);
    }
}

```

```

// Decreases value of key at index 'i' to new_val. It is assumed
that
// new_val is smaller than harr[i].
void MinHeap::decreaseKey(int i, int new_val)
{
    harr[i] = new_val;
    while (i != 0 && harr[parent(i)] > harr[i])
    {
        swap(&harr[i], &harr[parent(i)]);
        i = parent(i);
    }
}

// Method to remove minimum element (or root) from min heap
int MinHeap::extractMin()
{
    if (heap_size <= 0)
        return INT_MAX;
    if (heap_size == 1)
    {
        heap_size--;
        return harr[0];
    }

    // Store the minimum value, and remove it from heap
    int root = harr[0];
    harr[0] = harr[heap_size-1];
    heap_size--;
    MinHeapify(0);

    return root;
}

// This function deletes key at index i. It first reduced value to
minus
// infinite, then calls extractMin()
void MinHeap::deleteKey(int i)
{
    decreaseKey(i, INT_MIN);
    extractMin();
}

```



```

}

// A recursive method to heapify a subtree with the root at given
index
// This method assumes that the subtrees are already heapified
void MinHeap::MinHeapify(int i)
{
    int l = left(i);
    int r = right(i);
    int smallest = i;
    if (l < heap_size && harr[l] < harr[i])
        smallest = l;
    if (r < heap_size && harr[r] < harr[smallest])
        smallest = r;
    if (smallest != i)
    {
        swap(&harr[i], &harr[smallest]);
        MinHeapify(smallest);
    }
}

// A utility function to swap two elements
void swap(int *x, int *y)
{
    int temp = *x;
    *x = *y;
    *y = temp;
}

// Driver program to test above functions
int main()
{
    MinHeap h(11);
    h.insertKey(3);
    h.insertKey(2);
    h.deleteKey(1);
    h.insertKey(15);
    h.insertKey(5);
    h.insertKey(4);
    h.insertKey(45);
    cout << h.extractMin() << " ";
    cout << h.getMin() << " ";
}

```

```
h.decreaseKey(2, 1);  
cout << h.getMin();  
return 0;  
}
```

Output

2 4 1

Applications of Heaps

- Heap Sort: [Heap Sort](#) uses Binary Heap to sort an array in $O(n \log n)$ time.
- Priority Queue: [Priority queues](#) can be efficiently implemented using Binary Heap because it supports `insert()`, `delete()` and `extractmax()`, `decreaseKey()` operations in $O(\log N)$ time. Binomial Heap and Fibonacci Heap are variations of Binary Heap. These variations perform union also efficiently.
- Graph Algorithms: The priority queues are especially used in Graph Algorithms like [Dijkstra's Shortest Path](#) and [Prim's Minimum Spanning Tree](#).
- Many problems can be efficiently solved using Heaps.
See following for example. a) [K'th Largest Element in an array](#). b) [Sort an almost sorted array/](#) c) [Merge K Sorted Arrays](#).

Related Links:

- [Coding Practice on Heap](#)
- [All Articles on Heap](#)
- [Quiz on Heap](#)

Binary Heap Operations | DSA Problem

 Comment

More info ▼

Advertise with us

Next Article >

Advantages and Disadvantages of
Heap

Similar Reads

Heap Data Structure

A Heap is a complete binary tree data structure that satisfies the heap property: for every node, the value of its children is greater than or equal to its own value. Heaps are usually used to implement priority...

🕒 2 min read

Introduction to Heap - Data Structure and Algorithm Tutorials

A Heap is a special tree-based data structure with the following properties: It is a complete binary tree (all levels are fully filled except possibly the last, which is filled from left to right). It satisfies either the...

🕒 15+ min read

Binary Heap

A Binary Heap is a complete binary tree that stores data efficiently, allowing quick access to the maximum or minimum element, depending on the type of heap. It can either be a Min Heap or a Max...

🕒 13 min read

Advantages and Disadvantages of Heap

Advantages of Heap Data Structure Time Efficient: Heaps have an average time complexity of $O(\log n)$ for inserting and deleting elements, making them efficient for large datasets. We can convert any array to a...

🕒 2 min read

Time Complexity of building a heap

Consider the following algorithm for building a Heap of an input array A. A quick look over the above implementation suggests that the running time is $O(n * \lg(n))$ since each call to Heapify costs $O(\lg(n))$ an...

🕒 2 min read

Applications of Heap Data Structure

Heap Data Structure is generally taught with Heapsort. Heapsort algorithm has limited uses because Quicksort is better in practice. Nevertheless, the Heap data structure itself is enormously used. Priority...

🕒 2 min read

Comparison between Heap and Tree

What is Heap? A Heap is a special Tree-based data structure in which the tree is a complete binary tree. Types of Heap Data Structure: Generally, Heaps can be of two types: Max-Heap: In a Max-Heap the key...

🕒 3 min read

When building a Heap, is the structure of Heap unique?

What is Heap? A heap is a tree based data structure where the tree is a complete binary tree that maintains the property that either the children of a node are less than itself (max heap) or the children a...

🕒 4 min read

Some other type of Heap



Easy problems on Heap

